IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

A Profile Guided Register Allocation Prototype with Greedy Packing and Potential Guided Local Search

P R Suhruth, Utkarsh Agrawal, and Prakash Kumar Singh

Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India

P R Suhruth: suhruth.pr2024@vitstudent.ac.in

Utkarsh Agrawal: utkarsh.agrawal2024@vitstudent.ac.in

Prakash Kumar Singh: prakash.kumarsingh2024@vitstudent.ac.in

Corresponding author: Utkarsh Agrawal (utkarsh.agrawal2024@vitstudent.ac.in).

ABSTRACT Register allocation maps virtual registers onto a finite physical register file and spills the remainder. Execution profiles can change which spilled values are most expensive under a frequency-weighted cost model. This paper presents PGARA, a batch prototype for a restricted three-address representation. The prototype builds a control-flow graph, computes backward may-liveness, constructs an interference graph with one unsplit live range per virtual register, and evaluates spill assignments using caller-supplied block counts. Static Greedy orders ranges by profile-independent spill-site weight; Profile-Weighted Greedy uses costs summed across profiles; Potential-Guided Local Search accepts feasible single-range moves only when an average-to-worst-profile objective strictly decreases. Recomputed results on eight built-in synthetic cases and the dashboard sample show that profile weighting changes the spill decision from an estimated cost of 101 to 8 in a hot-block case. At a worst-profile weight of one, local search reduces the worst-profile estimate from 19 to 10 while increasing the summed estimate from 19 to 20. Additional reported results cover four computational kernels and two conventional allocation baselines. Their interpretation is limited by incomplete scoring and configuration details. The study distinguishes estimated spill operations from generated-code measurements and makes no claim of global optimality or improved application runtime.

INDEX TERMS compiler optimization, interference graph, local search, profile-guided optimization, register allocation, spill cost

I INTRODUCTION

Register allocation is a late compiler pass that determines which values remain in a limited physical register file. Values needed at the same program point cannot occupy the same register. When register pressure exceeds the available capacity, an allocator selects values for spilling to memory. Reloads and stores can increase instruction count and execution cost, making the choice of spilled values consequential.

A static count of definitions and uses treats hot and cold blocks alike. Execution-profile information can reverse an allocation preference because a small number of sites in frequently executed blocks may dominate many sites on rarely executed paths. This motivates an explicit account of the spill sites, the supplied counts, and the objective used to compare assignments. Profile information alone does not determine the cost of generated machine code: spill placement, splitting, rematerialization, and target constraints also matter.

PGARA expands to profile-guided adaptive register allocation. Here, adaptation means a batch response to user-supplied programs, register budgets, and profiles; it is not an online controller. The prototype constructs an interference graph from a restricted three-address representation and compares Static Greedy, Profile-Weighted Greedy, and Potential-Guided Local Search. The objective is to examine how allocation choices change estimated spill costs under supplied profiles, with an inspectable pipeline and dashboard.

The contributions are a restricted-IR parsing and analysis pipeline; three allocation procedures with explicit ordering or search objectives; a cost-estimation and visualization workflow; and an evaluation on the implemented synthetic cases and dashboard sample. Additional computational-kernel results are reported separately. These contributions concern allocation assignments and estimated spill operations. The public prototype does not rewrite spilled ranges into executable loads and stores, and the evidence does not establish improved compiled-program performance.

Two limitations motivate the comparison. First, a single aggregate spill weight hides disagreements among profiles.

Second, a search that moves only one live range at a time can miss an improvement requiring coordinated reassignment. A lower worst-profile estimate can therefore coexist with a higher aggregate estimate, while termination of the local search cannot establish global optimality.

II RELATED WORK

Register allocation has been studied through graph coloring, linear scan, profile-assisted spill selection, and search or learning methods. These approaches differ in their interference representation, handling of spill code, and evaluation objective. The relevant comparison with PGARA concerns both the method and the evidence used to assess it.

A Classical allocation methods

Chaitin and colleagues formulated allocation as graph coloring: nodes represent computed quantities, and edges connect simultaneously live values [1]. Their implementation in an optimizing PL/I compiler established the practical relevance of coloring. Chaitin subsequently integrated spill selection with graph coloring [2]. PGARA also uses interference constraints, but assigning a range to spill ends its transformation pipeline; no machine-code rewriting or subsequent reallocation is performed.

Briggs, Cooper, and Torczon developed optimistic coloring and extended rematerialization [3]. Optimistic coloring postpones a final spill decision until color selection, while rematerialization can reconstruct inexpensive values instead of reloading them. Their compiler experiments examine spill-code behavior on FORTRAN programs. PGARA has neither rematerialization nor a rewrite-and-retry phase, so its whole-range cost estimates do not measure the same effect.

George and Appel interleave coloring reductions with conservative coalescing to remove more copies while preserving colorability [4]. This addresses a limitation of performing conservative coalescing in isolation. PGARA does not coalesce copies, and moving a kept range from one physical register to another does not change its objective. A lower copy count in a production allocator therefore cannot be inferred from a lower PGARA spill estimate.

Poletto and Sarkar introduced linear scan as a fast alternative to graph coloring, traversing live intervals and tracking those active at the current position [5]. Traub, Holloway, and Smith compared linear scan with a graph-coloring allocator in Machine SUIF and examined changes that improve code quality while preserving a linear sweep [6]. Their results expose a compilation-time versus code-quality trade-off. PGARA's one-pass greedy methods still construct and use an interference graph; they are not linear-scan implementations merely because they visit ranges once.

Wimmer and Mössenböck extended linear scan in the HotSpot client compiler with lifetime holes, interval splitting, fixed target constraints, and elimination of unnecessary spill stores [7]. Their evaluation uses executable Java benchmarks. Pereira and Palsberg model register files and variables as puzzles, naturally representing precoloring and aliasing, and evaluate allocation on SPEC CPU2000 [8]. Both studies show why realistic allocation involves more than selecting a complete range to spill. PGARA's scalar, unsplit representation does not capture these capabilities.

B Profile guided spill decisions

Profile-assisted allocation, investigated by Kreahling and Norris [9], uses execution behavior to inform spill-cost decisions. The motivating issue is that static occurrences are an incomplete proxy for dynamic cost. PGARA adopts this frequency-sensitive view but accepts complete block-count maps from the caller rather than collecting profiles itself. It also allows several profiles to disagree, making the aggregation rule part of the experimental question.

Burroughs uses branch probabilities to estimate distance to the next use and block frequencies to optimize post-allocation spill placement [10]. The distinction between selecting a spill and placing its instructions is central: placement opportunities can change as allocation proceeds. PGARA instead sums weighted use and live-definition sites for each spilled range. It does not insert spill instructions, so its estimates cannot be equated with dynamically executed loads and stores in that study.

LLVM's greedy allocator provides a production example of the interaction among spill weights, eviction, and global live-range splitting [11]. A value used intensively in a hot loop can remain in a register there while spilling elsewhere. PGARA represents each virtual register as one range, so it cannot make that regional choice. Consequently, common block-frequency inputs would still not make the two allocators' output metrics interchangeable.

C Search based allocation and the research gap

RL4ReAl embeds reinforcement learning within LLVM and evaluates generated code for x86 and AArch64 [12]. Its architecture constraints and compiler integration make its results different from an assignment-map comparison. The interference-graph dataset of da Silva, Senefonte, and Attrot supports training and evaluation of learning methods [13], whereas PGARA neither trains a model nor uses that dataset. BitEA explores allocation through a bit-encoded evolutionary algorithm [14]; its population-based search differs from PGARA's deterministic single-range neighborhood.

The gap addressed by PGARA is an inspectable comparison under an explicit multi-profile estimated-cost objective. Classical methods prioritize feasible assignments and generated-code quality, while compiler evaluations also consider allocation time and executable performance. PGARA isolates supplied profile counts and the average-to-worst trade-off. This is a narrower implementation and evaluation scope, not evidence that the prototype outperforms

production allocators. Its additional kernel comparisons must be interpreted with their configuration and scoring limitations.

III PROPOSED WORK AND METHODOLOGY

A Problem formulation

The input consists of a program in a restricted three-address language, an integer physical-register budget k, and zero or more execution profiles. Instructions assign constants, copy registers, or perform addition, subtraction, multiplication, and division. Terminators are conditional branches of the form if register goto label, unconditional jumps, and returns. A comparison such as if t3 > 10 goto B1 is rejected. The false successor of a conditional branch is the next block in source order, and the last block must terminate with a return or jump.

Every profile maps every block label to a nonnegative integer execution count. Missing labels and unknown labels are rejected rather than supplied with default counts. The analyzer validates the map's form, but does not establish that its counts describe a feasible concrete execution or a representative workload. The profiles are assumptions of the cost comparison.

Let R contain the virtual registers live at any instruction boundary. Each has one unsplit live range; its allocation identifier equals its register name. An assignment A maps each range to a physical register p_i, where $0 \le i < k$, or to spill. JSON null denotes spill. An assignment is feasible when no pair of interfering ranges shares a physical register. The service accepts $0 \le k \le 256$. At $k = 0$ all ranges spill; insufficient registers otherwise produce spills rather than an input error.

B System architecture and modules

Figure 1 separates the analysis library, HTTP service, and dashboard. The library parses text or structured blocks, validates input, builds a control-flow graph (CFG), computes liveness, constructs interference, and evaluates allocation. The service exposes health, a sample program, allocation, and a synthetic benchmark endpoint. The React dashboard shows returned assignments, an interference graph, estimated costs, and search traces.

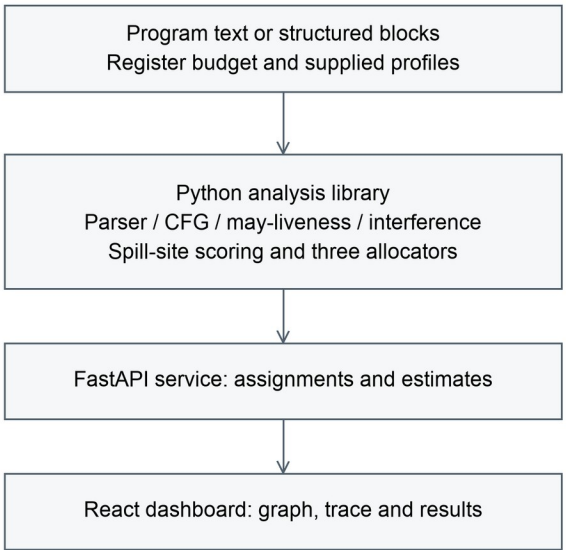


FIGURE 1. PGARA system architecture. The output consists of assignments and estimated spill costs.

The parser accepts one instruction per line, # comments, and block headers of the form block name:. Structured input supports const, mov, add, sub, mul, div, br, jmp, and ret. Exactly one of source text or structured blocks is required. Service limits are 32 profiles, 500 blocks, 5,000 instructions, and 100,000 source characters. Unknown opcodes, extra JSON fields, invalid branches, and negative profile counts are rejected.

The dashboard has two data modes. Sample mode displays a fixed illustrative fixture; editing the program does not recompute it. Backend mode requests an analysis and reports errors without replacing a failed request with fixture data. The register slider covers 1–16, while the API accepts the larger budget range. Allocation requests use the robustness slider as the trade-off, an all-spilled start, and a 200-move budget. When a benchmark request omits optional parameters, each built-in case uses its configured trade-off and iteration budget.

The local backend address is http://127.0.0.1:8741. Cross-origin requests are allowed from localhost and 127.0.0.1 without credentials. Profile-Weighted Greedy and Potential-Guided Local Search require at least one profile. An all-algorithm request without profiles runs Static Greedy and marks the other two as skipped. The API retains the identifier potential_game for compatibility; this paper uses Potential-Guided Local Search to describe that method and does not claim a formal potential-game result.

C Allocation flow and synthetic programs

The analysis order is shown in Fig. 2. Liveness must reach a fixpoint before interference and spill sites are constructed. A single backward pass is insufficient for loops. Assignment validity is checked after allocation, independently of the cost objective.

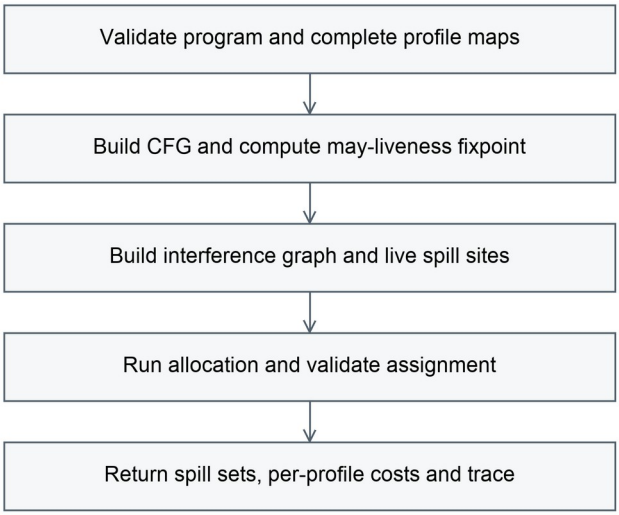


FIGURE 2. Allocation flow. Complete supplied profile maps are validated before scoring.

The built-in suite comprises eight synthetic cases, with stored parameters shown in Table 1. Sequential reuse tests sharing one register among nonoverlapping ranges. The diamond exercises a branch and join; its false edge enters the following right block. The loop exercises a back edge and the liveness fixpoint. The profile-versus-static case places three updates of a in a cold block and one use of b in a hot block. The two trade-off cases use the same program at different values of λ. Zero-register and insufficient-register cases test spilling under constrained capacity.

TABLE 1. Built-in synthetic cases and stored search parameters.

Case	k	λ	Budget
sequential reuse	1	1/2	10
diamond	1	1/2	20
loop	1	1/2	20
profile versus static	1	0	10
game average	1	0	10
game worst	1	1	10
zero registers	0	1/2	5
insufficient registers	1	1/2	20

Figures 3 and 4 show the trade-off program and its interference graph. The profiles skew and quiet_b assign counts to def_a, def_b, use_a, use_b, and exit of (2, 9, 3, 10, 2) and (2, 0, 3, 0, 2), respectively. With unit weights, spilling a costs 10 in either profile; spilling b costs 19 in skew and 0 in quiet_b. The new value defined by b = b + 1 is dead, so that instruction contributes a use but no live-definition store site.

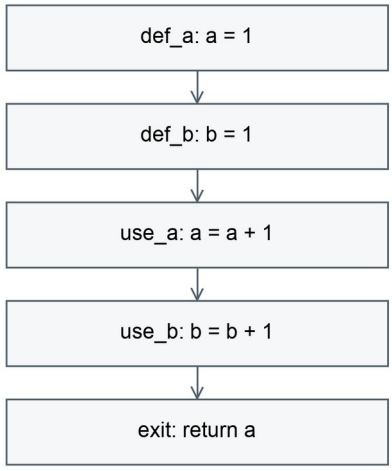


FIGURE 3. CFG of the built-in trade-off program. Each edge is an unconditional jump.

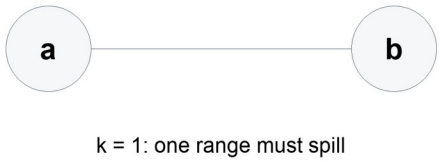


FIGURE 4. Interference graph for the trade-off program. Ranges a and b cannot share a register.

D Liveness and spill sites

PGARA uses backward may-liveness. A block's live-out set is the union of its successors' live-in sets. Within a block, each destination is removed and each used virtual register is added while walking backward. Block live-in and live-out sets are iterated until unchanged. The convergence guard raises an error after more than $2BN + 2$ iterations, using $N = 1$ when no registers are mentioned, where B is the block count and N otherwise counts mentioned virtual registers. This is an implementation guard, not a complexity theorem.

After convergence, the backward transfer is recorded at instruction boundaries. Two ranges interfere if they occur together in a boundary live set. A destination live immediately after an instruction is a store site; a dead destination is omitted from store sites. Each distinct register used in an instruction contributes one use site, even if it appears twice as an operand. Undefined-use names are reported when no definition exists anywhere in the function. This check does not establish initialization on every path.

The analysis is path-insensitive. Unions merge information from different successors without proving joint path feasibility. Additional conservative interference can therefore restrict legal assignments and may lead to spills avoidable under more precise analysis. The current evaluation does not isolate or quantify this effect. Conservative edges are not

treated as analysis errors, and path-sensitive liveness is not implemented.

The response includes CFG edges, per-instruction live sets, interference edges and degrees, use and live-definition sites, undefined uses, and dead definitions. Local-search results also include accepted moves and the stopping reason. The dashboard visualizes returned graph edges; it does not perform a second liveness analysis.

E Cost model and objective

All costs are dimensionless estimates. Let U(r) and D(r) be the use sites and live-definition sites of range r; dead definitions are excluded. Let w_L and w_S be nonnegative integer load and store weights, and let $f_m(b)$ be the supplied count of block b in profile m. The service defaults to $w_L = w_S = 1$ and permits weights from 0 through 1,000. Profile-dependent expressions require $M \geq 1$ profiles.

$$C_{static}(r) = w_L|U(r)| + w_S|D(r)| \quad (1)$$

$$C_m(r) = w_L \sum_{u \in U(r)} f_m(block(u)) + w_S \sum_{d \in D(r)} f_m(block(d)) \quad (2)$$

$$C_m(A) = \sum_{r:A(r)=spill} C_m(r) \quad (3)$$

$$W(r) = \sum_{m=1}^{M} C_m(r) \quad (4)$$

Equation (1) ignores execution profiles. Equation (2) weights each recorded site by its containing block's count. Equation (3) charges only spilled ranges. Profile-Weighted Greedy orders ranges by the per-range sum W(r), whereas Static Greedy uses $C_{static}(r)$. Costs are evaluated under the same profiles after either assignment, enabling a common estimate comparison.

$$\Phi_\lambda(A) = (1 - \lambda)\frac{1}{M}\sum_{m=1}^{M} C_m(A) + \lambda \max_m C_m(A) \quad (5)$$

The trade-off λ lies in [0, 1]. At λ = 0 the objective is average profile cost; at λ = 1 it is the worst-profile cost. The service converts the trade-off to a rational with denominator at most 10^6. Average, maximum, and candidate comparisons use exact rational arithmetic; the displayed floating-point objective is not used to decide moves.

$$\Phi_\lambda(A') < \Phi_\lambda(A) \quad (6)$$

$$feasible(r, p, A) \Leftrightarrow p \notin A(N(r)) \quad (7)$$

Spill is always feasible. In (7), A(N(r)) denotes the registers held by neighbors of range r. An interference edge requires joint membership in a single instruction-boundary live set. Let $\mathcal{L}$ contain each before-instruction and after-instruction live set separately:

$$\{r, q\} \in E \Leftrightarrow \exists L \in \mathcal{L}: r, q \in L,\ r \neq q \quad (8)$$

The model treats supplied block counts as proxies for dynamic events. It does not model actual spill placement, reuse of loaded values, splitting, coalescing, rematerialization, register classes, or target instruction costs. Two assignments with the same spilled ranges receive the same estimated cost under identical programs, profiles, and weights, even if their physical-register mappings differ. Thus the estimate is a decision metric, not measured memory traffic, code size, cycles, or runtime.

When both weights are zero, every objective is zero. Local search accepts no move from an all-spilled start because no strict decrease exists. Greedy packing can still assign registers because it does not require objective improvement. This corner case is distinct from a proof that an unspilled assignment has no execution overhead.

F Allocation algorithms

Static Greedy and Profile-Weighted Greedy use the shared packing procedure in Algorithm 1. Ranges are ordered by decreasing weight, with ties broken by ascending identifier. Each receives the lowest-numbered physical register unoccupied by an already assigned interfering neighbor, or spills if none is free. The pass does not reconsider earlier decisions. Neither ordering is a proof that the final objective is minimized.

ALGORITHM 1. One-pass greedy packing

Input: ranges R, edges E, register budget k, weight w(r).
1. Order R by decreasing w(r), then ascending range identifier.
2. For each r, collect registers held by assigned neighbors.
3. Assign the lowest free register; spill r if no register is free.
4. Validate the assignment; evaluate profile and static costs.
Static Greedy: $w(r) = C_{static}(r)$.
Profile-Weighted Greedy: w(r) = W(r).

Potential-Guided Local Search starts from a valid assignment and examines single-range moves to each physical register and spill. It keeps the feasible candidate with the lowest objective strictly below the current objective. Ranges are scanned by identifier and strategies in the order p0, p1, …, spill. An equal-valued candidate does not replace an earlier best candidate. Algorithm 2 and Fig. 5 describe the acceptance and stopping rules.

ALGORITHM 2. Potential-Guided Local Search

Input: R, E, k, profiles, λ, move budget B, valid start A.
1. For at most B accepted moves, set best = none.
2. Scan each range r and each alternative strategy s.
3. If feasible, form A′ by changing only A(r) to s.
4. Keep A′ only if Φλ(A′) < Φλ(A) and below the best so far.
5. If best is none, stop with no improving move.
6. Otherwise set A = best and record the accepted move.
7. On exhausting B, report budget exhaustion; validate A.

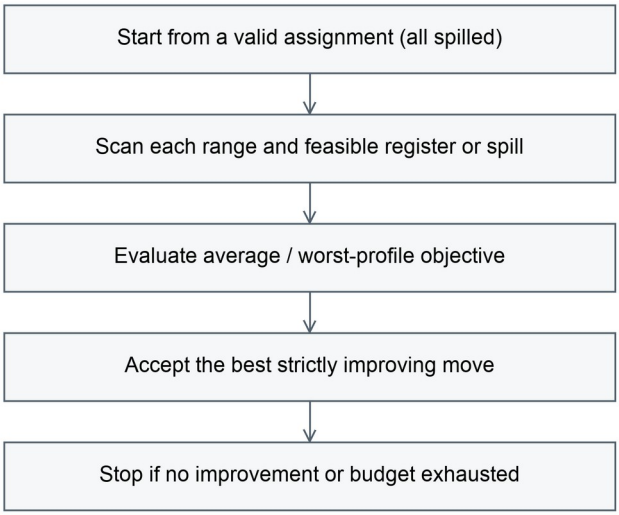


FIGURE 5. Potential-Guided Local Search. Acceptance requires a feasible strict improvement.

The default start spills all ranges. Optional starts use either greedy assignment. Because the model depends only on the spill set, a move between two physical registers leaves the objective unchanged. A maximal greedy packing may therefore be a stopping point even when a two-range exchange would reduce cost. The trade-off example at λ = 1 stops at spill b when initialized from Profile-Weighted Greedy, while the all-spilled start reaches spill a.

The restricted neighborhood may miss coordinated improvements such as pairwise swaps or eviction moves. These extensions are future work. Stopping with no improving move establishes only that no strict single-range improvement was found; budget exhaustion is weaker still. The algorithm is described as local search because a scalar potential objective and strict decreases alone do not establish a formal potential game or a global optimum.

G Conventional comparison baselines

The additional kernel comparison includes Linear Scan and Chaitin–Briggs graph coloring. Linear scan orders intervals by start position, expires inactive intervals, and maintains an active set occupying physical registers [5], [6]. A spill-victim rule resolves capacity conflicts. Chaitin–Briggs coloring simplifies low-degree nodes onto a stack, selects potential spill candidates when simplification stalls, and restores nodes in reverse order using optimistic color selection [2], [3].

These descriptions identify the algorithm families used in the reported comparison. Interval construction, spill-victim selection, and any rewrite-and-retry policy are material variant choices. The public PGARA revision examined for the reproducible experiments implements only the three primary methods. Its code does not provide the two additional baseline implementations, so their variant-specific behavior is not established by that revision.

H Benchmark programs and evaluation protocol

The evaluation distinguishes three input groups: eight built-in synthetic cases, the dashboard sample, and four computational kernels. The additional kernels are matrix multiplication, array summation, iterative Fibonacci computation, and conditional numeric computation. They represent arithmetic, recurrence, and control-flow patterns beyond the synthetic cases. Reported kernel results cover k = 2 and k = 4 for the primary methods and k = 2 for the five-method comparison.

A controlled comparison requires an identical IR program, register budget, profile map, and load/store weights for each algorithm. Assignment validity means satisfaction of the interference graph, not proof of equivalence between generated machine code and the original program. For kernel comparisons, the input translations, profile maps, and search configuration must also determine the meaning of the reported estimated costs. The limits of the available configuration information are discussed in Section IV.

I Spill code generation and validation

Spill-code generation is a transformation beyond identifying spilled ranges. A complete transformation would assign memory locations, insert loads before required uses and stores after definitions whose values must survive, and rewrite instructions using temporary registers within the target budget. Multiple uses, redefinitions, control-flow joins, and loaded-value reuse affect the inserted operations. Unlimited scratch registers cannot be assumed.

The public prototype stops at assignments and estimated costs. Spill-code insertion and executable validation are outside that implemented pipeline. A future evaluation should compare original and transformed program outputs, count inserted loads and stores and total instructions, and report dynamic execution measurements separately. Estimated cost, static inserted-instruction count, dynamic memory operations, and measured runtime are distinct quantities; one cannot be recovered from another without additional evidence.

IV IMPLEMENTATION AND RESULT ANALYSIS

A Tools and reproducibility

The analyzer is a Python package, version 0.1.0, exposed through FastAPI with Pydantic request and response models. The interface uses TypeScript, React, and Vite; automated backend tests use pytest. The public code is available in the PGARA repository [15]. Its README documents environment setup and the test command python -m pytest from backend/.

The built-in and dashboard estimates reported here were recomputed by calling pgara.engine.analyze at repository commit 90ac3af118bdb873dd1e49855a5f6ac6b1487248. The verification used Python 3.12.14. Load and store weights were 1, and local search used the all-spilled start. Backend inputs are defined in backend/pgara/samples.py; the dashboard program and profile counts are in src/features/pgara/types.ts. Each recomputed assignment passed the interference-constraint check. The commit identifies the verified code version without assuming a release tag.

No controlled runtime benchmark was conducted for the reproduced experiments. The service returns an allocation-time field, but its unit test checks only that the value is nonnegative. Such a check does not establish relative allocator speed. The evaluation of the reproduced inputs therefore focuses on assignments and estimated spill costs; no speedup over LLVM or GCC is inferred.

B Parameters and metrics

TABLE 2. Parameters of the reproduced allocation runs.

Parameter	Dashboard	Synthetic suite
Register count k	2, 3, 4, 5, 6	Per Table 1
Trade-off λ	3/5	Per Table 1
Load/store weights	1 / 1	1 / 1
Search start	All spilled	All spilled
Move budget	200	Per Table 1
Profiles	Three maps	Stored case maps

The dashboard block order is B0, B2, B1, B3. Its profiles Hot loop, Balanced, and Cold-heavy use counts (1000, 150, 850, 1000), (1000, 500, 500, 1000), and (1000, 800, 200, 1000). The label Hot loop is a profile name; the dashboard program itself has a branch rather than a back edge. The allocation trade-off is 3/5 and the move budget is 200. The synthetic suite instead retains each case's own settings.

TABLE 3. Metrics used for the reproduced experiments.

Metric	Definition
Summed estimate	Sum of C_m(A) over supplied profiles
Worst-profile estimate	Maximum C_m(A) over supplied profiles
Static estimate	Sum of C_static(r) for spilled ranges
Spill set	Identifiers assigned to spill
Allocation validity	No interfering pair shares a physical register
Spill-set agreement	Whether algorithms return the same spill set

A zero estimate is a cost-model outcome. It does not imply that execution has zero overhead. Spill-set agreement does not imply identical physical-register maps, and it is not a theorem that the algorithms agree on other programs. The estimates are comparative values under shared assumptions rather than measurements of application performance.

C Dashboard sample results

TABLE 4. Reproduced dashboard sample at λ = 3/5. All three methods share each row.

k	Sum	Worst	Static	Spilled ranges
2	16,550	5,850	7	c, d, t2, t4
3	3,000	1,000	1	d
4	0	0	0	None
5	0	0	0	None
6	0	0	0	None

At k = 2, all three algorithms spill c, d, t2, and t4. Per-profile estimates are 5,850, 5,500, and 5,200, summing to 16,550; the static estimate is 7. At k = 3, each spills only d, with cost 1,000 per profile and summed estimate 3,000. At k = 4, 5, and 6, the spill set is empty and every profile estimate is zero. The sample has ten live ranges: six, nine, and ten are kept at these respective budgets.

Matching sums follow from matching spill sets. For example, at k = 6 Static Greedy and the other two methods differ in their placement of t1, t3, and t5, yet all have zero spill cost. The cost model intentionally does not distinguish those kept-register mappings. The fixed sample-mode fixture values 4,200, 1,800, and 1,200 are excluded from these experimental tables.

D Synthetic cases and the search objective

TABLE 5. Synthetic cases with a shared spill set across all three methods.

Case	k	Sum	Static	Spilled ranges
sequential reuse	1	0	0	None
diamond	1	3	3	b
loop	1	40	4	s
zero registers	0	6	6	a, b, c
insufficient registers	1	8	8	b, c, d, f

Five cases produce identical spill sets for all three methods under the stored settings. Sequential reuse spills nothing with one register; the diamond spills b; the loop spills s. At zero

registers a, b, and c spill. The insufficient-register case spills b, c, d, and f. These cases establish behavior under the implemented graph and scoring rules, not an advantage of local search.

TABLE 6. Reproduced comparison at k = 1. SG: Static Greedy; PWG: Profile-Weighted Greedy; PGLS: Potential-Guided Local Search.

Case	λ	Method	Sum	Worst	Spill
Profile vs static	0	SG	101	101	b
Profile vs static	0	PWG	8	8	a
Profile vs static	0	PGLS	8	8	a
Average case	0	SG	19	19	b
Average case	0	PWG	19	19	b
Average case	0	PGLS	19	19	b
Worst case	1	SG	19	19	b
Worst case	1	PWG	19	19	b
Worst case	1	PGLS	20	10	a

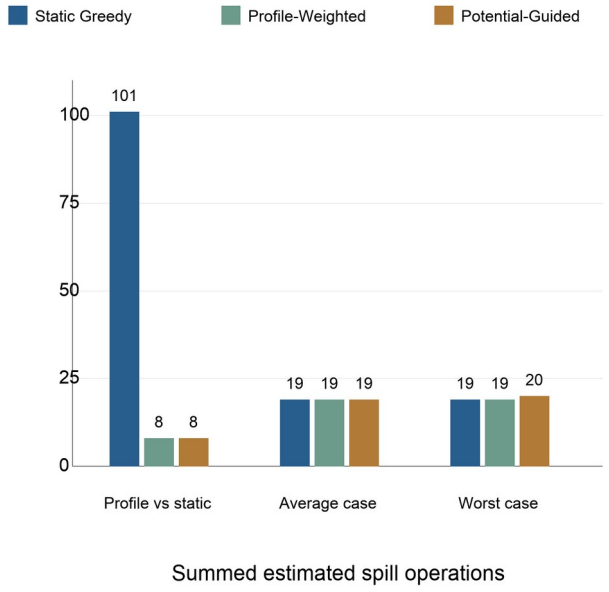


FIGURE 6. Reproduced summed estimated spill operations for three synthetic cases. Each uses its stored trade-off.

In profile_versus_static, Static Greedy spills b with summed and worst-profile cost 101. Profile-Weighted Greedy and local search spill a with cost 8. The case has one profile, so its summed and worst-profile values coincide. Profile weighting therefore changes the selected range under the supplied hot-block counts.

In game_average at λ = 0, all three methods spill b: costs are (19, 0), the summed estimate is 19, the worst-profile estimate is 19, and the average objective is 9.5. In game_worst at λ = 1, both greedy methods retain that assignment. Local search instead spills a: costs are (10, 10), giving summed cost 20 and worst-profile cost 10. The selected objective improves from 19 to 10 although the sum rises by one. This is a configuration-dependent robustness trade-off, not general dominance over greedy allocation.

For this two-range example, spilling a has objective 10 for every λ. Spilling b has objective $9.5(1 - \lambda) + 19\lambda = 9.5 + 9.5\lambda$. The crossover is $\lambda = 1/19$. Below it the all-spilled search keeps a and spills b; above it the search keeps b and spills a. At the exact tie, deterministic scan order keeps a first and hence spills b. This analytic comparison concerns only the two feasible one-spill outcomes of the example.

E Evaluation on representative programs

The four computational kernels extend the reported program coverage. Table 7 lists the supplied results for the three primary methods at k = 2 and k = 4. Table 8 compares five methods at k = 2, and Table 9 reports the corresponding supplied spill counts. These kernel results are separated from the synthetic and dashboard results recomputed from the identified public code revision.

TABLE 7. Reported register-allocation outcomes on four computational kernels. Costs are estimates; configuration limitations are discussed in Section IV-F.

Program	Algorithm	k	Reported spill set	Estimated cost	Valid
Matrix multiplication	Static Greedy	2	a, c, t2, t4	12,400	Yes
Matrix multiplication	Profile-Weighted Greedy	2	c, t4	8,200	Yes
Matrix multiplication	Potential-Guided Local Search	2	c, t4	7,900	Yes
Matrix multiplication	Static Greedy	4	t4	2,100	Yes
Matrix multiplication	Profile-Weighted Greedy	4	t4	1,500	Yes
Matrix multiplication	Potential-Guided Local Search	4	t4	1,350	Yes
Array summation	Static Greedy	2	sum, t2	6,400	Yes
Array summation	Profile-Weighted Greedy	2	t2	3,200	Yes
Array summation	Potential-Guided Local Search	2	t2	2,950	Yes
Array summation	Static Greedy	4	None	0	Yes
Array summation	Profile-Weighted Greedy	4	None	0	Yes
Array summation	Potential-Guided Local Search	4	None	0	Yes
Iterative Fibonacci	Static Greedy	2	prev, t3	5,800	Yes
Iterative Fibonacci	Profile-Weighted Greedy	2	t3	3,600	Yes
Iterative Fibonacci	Potential-Guided Local Search	2	t3	3,400	Yes
Iterative Fibonacci	Static Greedy	4	None	0	Yes
Iterative Fibonacci	Profile-Weighted Greedy	4	None	0	Yes
Iterative Fibonacci	Potential-Guided Local Search	4	None	0	Yes
Conditional numeric computation	Static Greedy	2	x, t2	7,200	Yes
Conditional numeric computation	Profile-Weighted Greedy	2	t2	4,100	Yes
Conditional numeric computation	Potential-Guided Local Search	2	t2	3,850	Yes
Conditional numeric computation	Static Greedy	4	None	0	Yes
Conditional numeric computation	Profile-Weighted Greedy	4	None	0	Yes
Conditional numeric computation	Potential-Guided Local Search	4	None	0	Yes

At k = 2, the reported estimated costs for Potential-Guided Local Search are 7,900 for matrix multiplication, 2,950 for array summation, 3,400 for iterative Fibonacci, and 3,850 for conditional computation. At k = 4, the latter three programs have zero reported estimated cost for all three methods, while matrix multiplication retains a reported spill. These observations concern the supplied program configurations and do not establish behavior on a recognized compiler benchmark suite.

TABLE 8. Reported estimated spill costs for five strategies at k = 2.

Program	k	Static Greedy	Profile-Weighted	Potential-Guided	Linear Scan	Chaitin-Briggs
Matrix multiplication	2	12400	8200	7900	9100	7600
Array sum	2	6400	3200	2950	3500	2800
Fibonacci	2	5800	3600	3400	3900	3300
Conditional computation	2	7200	4100	3850	4400	3700

TABLE 9. Reported spilled-range counts for five strategies at k = 2. Counts require reconciliation with the spill sets in Table 7.

Program	k	Static Greedy	Profile-Weighted	Potential-Guided	Linear Scan	Chaitin-Briggs
Matrix multiplication	2	4	3	3	3	2
Array sum	2	3	2	2	2	2
Fibonacci	2	3	2	2	2	2
Conditional computation	2	3	2	2	2	2

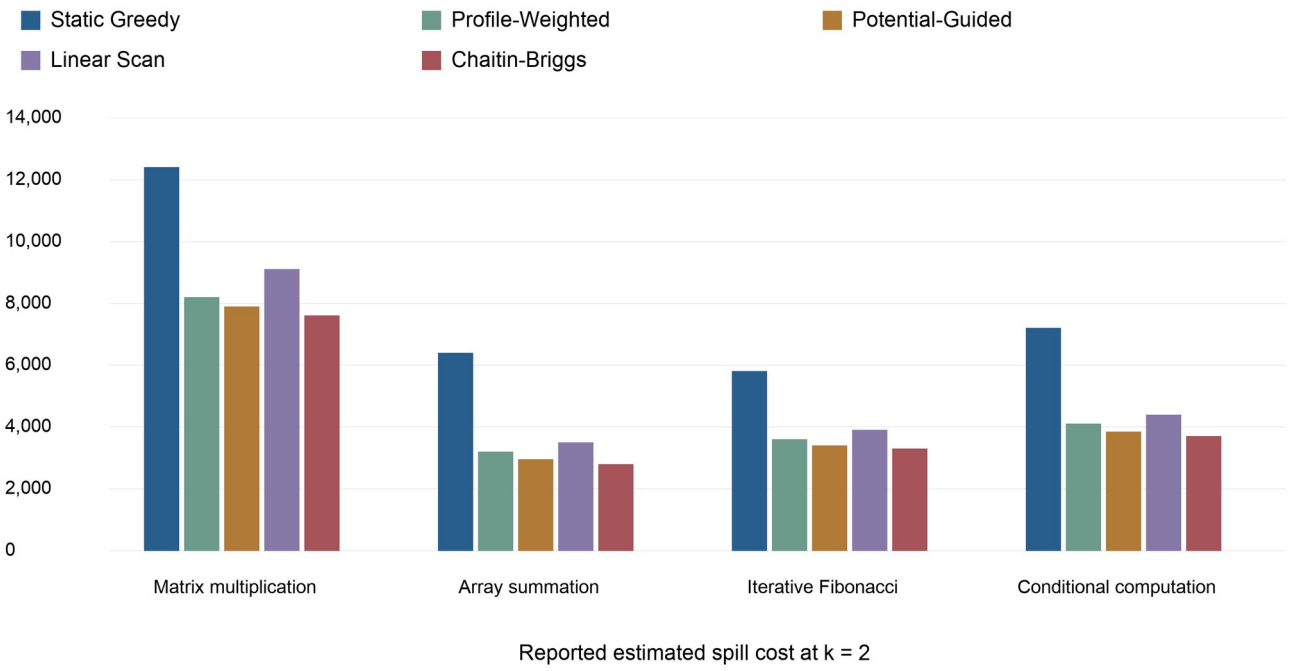


FIGURE 7. Reported estimated spill costs for the four kernels at k = 2. The plot reproduces Table 8; it is not an execution-time comparison.

In the reported five-strategy cost table, Chaitin–Briggs has the lowest estimate for each kernel, followed by Potential-Guided Local Search. Linear Scan lies between Profile-Weighted Greedy and Static Greedy in each case. Spill count alone is not enough to explain this ordering because the frequencies and costs of spilled ranges matter. Interpretation of these values also requires the consistency checks described next; the ranking does not establish an advantage of the verified local-search implementation under a common cost model.

F Consistency and limits of the kernel results

The reported kernel tables require additional configuration information before a common-objective comparison can be established. Table 7 assigns different estimated costs to identical spill sets: for example, at k = 2 the matrix-multiplication PWG and PGLS rows both spill c and t4 but report 8,200 and 7,900. At k = 4 all three matrix-multiplication rows spill t4 but report 2,100, 1,500, and 1,350. Equations (1)–(4) assign identical costs to identical spill sets under a common program, profiles, and weights. Thus these differences require

distinct scoring settings, a different cost model, or corrected table entries.

The reported counts also differ from the listed spill sets. At k = 2, Table 7 lists two spilled ranges for matrix-multiplication PWG and PGLS, while Table 9 reports three. For array summation, Fibonacci, and conditional computation, the listed SG sets have two ranges and the PWG/PGLS sets have one, whereas Table 9 reports three and two. The cost and count tables are retained as reported outcomes, but these inconsistencies prevent attributing their differences to allocator quality under the stated model.

The kernel IR translations, profile maps, search trade-offs and budgets, and baseline implementation variants are not established by the reproducible public revision. Consequently, the kernel results do not independently validate the three primary methods or demonstrate a controlled five-method advantage. The verified quantitative conclusions remain those of the reproduced synthetic and dashboard experiments.

G Interpretation and evaluation limits

The current pipeline does not insert spill code. Therefore, the reproduced estimate tables cannot establish the number of inserted loads or stores, total generated instructions, dynamic memory traffic, application runtime, or processor cycles. A separate table of such quantities requires a documented transformation and measurement protocol, including correctness checks, hardware and software details, inputs, repetitions, and variability. Estimated spill cost is not a substitute for those measurements.

Allocation outcomes also depend on conservative may-liveness. Extra interference from control-flow merging can reduce feasible choices, but no experiment here measures the number of spills specifically attributable to path insensitivity. The reported costs are neither lower bounds for a more precise analysis nor evidence that every conservative edge is avoidable.

The public prototype does not provide a comparison against LLVM or GCC, a controlled timing study, or a broad sweep of load/store weights and search budgets. The eight synthetic cases and dashboard sample expose specific decisions rather than establish generalization to production workloads. The kernel reports broaden the listed workloads but retain the scoring and reproducibility limitations described above.

V CONCLUSION AND FUTURE WORK

PGARA implements a restricted three-address analysis pipeline, an interference graph with one range per virtual register, three allocation procedures, and a dashboard for assignments and estimated spill costs. Static Greedy uses profile-independent site weights, Profile-Weighted Greedy uses costs summed across supplied profiles, and Potential-Guided Local Search accepts feasible single-range moves that strictly decrease an average-to-worst-profile objective.

The reproduced dashboard results agree across all three methods: summed estimates are 16,550 at two registers, 3,000 at three, and zero at four through six. Across the eight synthetic cases, profile weighting changes one hot-block decision from cost 101 to 8. The worst-profile search case improves the worst estimate from 19 to 10 while increasing the sum from 19 to 20. These results establish a specific profile-sensitive trade-off, not universal superiority of local search or improved generated-code execution.

Additional results cover four kernels and two conventional baselines, but their spill-set costs and spill counts require reconciliation before a common-model allocation comparison is supported. The primary implementation remains limited by unsplit ranges, path-insensitive liveness, trusted supplied profiles, and a single-range search neighborhood. It does not generate or execute spill code.

Future work includes spill-code generation with semantic validation, comparison of estimates against inserted and dynamically executed loads and stores, and measurements of code size and execution cycles. Repeated allocator-runtime trials should use a documented hardware and software configuration and report summary statistics and variability. More precise liveness, live-range splitting, rematerialization, pairwise swaps and eviction moves, and exact optimization on small instances would help determine the sources and limits of allocation quality.

AUTHOR CONTRIBUTIONS

P R Suhruth contributed to backend development and API implementation. Utkarsh Agrawal contributed to user interface and user experience design and software testing. Prakash Kumar Singh contributed to sample program collection and verification. All three authors contributed to the preparation and revision of the manuscript.

ACKNOWLEDGMENT

The authors acknowledge Bhuvaneswari M for her guidance and support during the development of the PGARA project.

AI disclosure: OpenAI Codex was used to assist with drafting and revising Sections I–V, declarations, captions, and reference formatting, and to support consistency checks of the reported results. The authors are responsible for reviewing and verifying the technical content, experimental results, references, and final wording.

FUNDING

This research received no external funding.

COMPETING INTERESTS

The authors declare that they have no competing interests.

DATA AND CODE AVAILABILITY

The PGARA prototype source code, dashboard, automated tests, and built-in sample inputs are publicly available at https://github.com/Sith-Bored/pgara-insight-dash. Commit 90ac3af118bdb873dd1e49855a5f6ac6b1487248 identifies the implementation used to recompute the synthetic and dashboard estimates. The synthetic inputs are in backend/pgara/samples.py and the dashboard input and profiles are in src/features/pgara/types.ts. The kernel tables are reported separately and are not reproduced by those built-in inputs.

REFERENCES

[1] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein, "Register allocation via coloring," Computer Languages, vol. 6, no. 1, pp. 47–57, 1981, doi: 10.1016/0096-0551(81)90048-5.

[2] G. J. Chaitin, "Register allocation & spilling via graph coloring," in Proc. SIGPLAN Symp. Compiler Construction, 1982, pp. 98–101, doi: 10.1145/800230.806984.

[3] P. Briggs, K. D. Cooper, and L. Torczon, "Improvements to graph coloring register allocation," ACM Trans. Program. Lang. Syst., vol. 16, no. 3, pp. 428–455, 1994, doi: 10.1145/177492.177575.

[4] L. George and A. W. Appel, "Iterated register coalescing," ACM Trans. Program. Lang. Syst., vol. 18, no. 3, pp. 300–324, 1996, doi: 10.1145/229542.229546.

[5] M. Poletto and V. Sarkar, "Linear scan register allocation," ACM Trans. Program. Lang. Syst., vol. 21, no. 5, pp. 895–913, 1999, doi: 10.1145/330249.330250.

[6] O. Traub, G. Holloway, and M. D. Smith, "Quality and speed in linear-scan register allocation," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation, 1998, pp. 142–151, doi: 10.1145/277650.277714.

[7] C. Wimmer and H. Mössenböck, "Optimized interval splitting in a linear scan register allocator," in Proc. ACM/USENIX Int. Conf. Virtual Execution Environments, 2005, pp. 132–141, doi: 10.1145/1064979.1064998.

[8] F. M. Q. Pereira and J. Palsberg, "Register allocation by puzzle solving," in Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation, 2008, pp. 216–226, doi: 10.1145/1375581.1375609.

[9] W. C. Kreahling and C. Norris, "Profile assisted register allocation," in Proc. ACM Symp. Applied Computing, 2000, pp. 774–781, doi: 10.1145/338407.338562.

[10] N. Burroughs, "Register allocation and spilling using the expected distance heuristic," Software: Practice and Experience, vol. 46, no. 11, pp. 1499–1523, 2016, doi: 10.1002/spe.2393.

[11] J. S. Olesen, "Greedy register allocation in LLVM 3.0," LLVM Project Blog, Sep. 18, 2011. [Online]. Available: https://blog.llvm.org/2011/09/greedy-register-allocation-in-llvm-30.html. Accessed: Oct. 28, 2026.

[12] S. VenkataKeerthy, S. Jain, A. Kundu, R. Aggarwal, A. Cohen, and R. Upadrasta, "RL4ReAl: Reinforcement learning for register allocation," in Proc. ACM SIGPLAN Int. Conf. Compiler Construction, 2023, doi: 10.1145/3578360.3580273.

[13] P. Z. da Silva, H. C. de M. Senefonte, and W. Attrot, "Interference graph dataset for machine learning-based register allocation," IEEE Access, vol. 12, pp. 157574–157586, 2024, doi: 10.1109/ACCESS.2024.3481358.

[14] G. S. Terci, E. Abdulhalik, A. A. Bayrakci, and B. Boz, "BitEA: BitVertex evolutionary algorithm to enhance performance for register allocation," IEEE Access, vol. 12, pp. 115497–115514, 2024, doi: 10.1109/ACCESS.2024.3446596.

[15] PGARA project, "pgara-insight-dash," GitHub repository, commit 90ac3af118bdb873dd1e49855a5f6ac6b1487248. [Online]. Available: https://github.com/Sith-Bored/pgara-insight-dash. Accessed: Oct. 28, 2026.

AUTHOR BIOGRAPHIES

P R Suhruth is affiliated with the Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. For the PGARA project, P R Suhruth contributed to backend and API development.

Utkarsh Agrawal is affiliated with the Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. For the PGARA project, Utkarsh Agrawal contributed to UI/UX design and software testing and serves as the corresponding author.

Prakash Kumar Singh is affiliated with the Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India. For the PGARA project, Prakash Kumar Singh contributed to sample program collection and verification.